

Отчёт по лабораторной работе №6

**Установка и настройка системы управления базами данных
MariaDB**

Цыкунова Екатерина Михайловна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Установка и первичная настройка MariaDB	6
2.2	Конфигурация кодировки символов UTF-8	14
2.3	Создание базы данных addressbook	20
2.4	Создание и восстановление резервных копий базы данных	26
2.5	Автоматизация установки и настройки MariaDB средствами Vagrant	28
3	Вывод	30
4	Контрольные вопросы	31

Список иллюстраций

2.1	Установка пакетов MariaDB и необходимых зависимостей	7
2.2	Просмотр конфигурационных файлов MariaDB	9
2.3	Запуск MariaDB, включение автозапуска и проверка порта 3306 .	10
2.4	Установка пароля администратора MariaDB	11
2.5	Завершение настройки безопасности MariaDB	12
2.6	Просмотр справки интерактивной оболочки MariaDB	13
2.7	Просмотр системных баз данных MariaDB	14
2.8	Статус MariaDB до изменения кодировки	17
2.9	Содержимое конфигурационного файла utf8.cnf	19
2.10	Статус MariaDB после настройки кодировки UTF-8	20
2.11	Создание базы addressbook, таблицы city и добавление данных .	22
2.12	Создание пользователя emcikunova и назначение прав на addressbook	24
2.13	Проверка базы addressbook и таблицы city с помощью mysqlshow	26
2.14	Создание и восстановление резервных копий базы addressbook .	28
2.15	Подготовка файлов для автоматической настройки MariaDB . . .	28
2.16	Скрипт автоматической установки и настройки MariaDB	29

Список таблиц

1 Цель работы

Приобретение практических навыков по установке и конфигурированию системы управления базами данных на примере программного обеспечения MariaDB.

2 Выполнение лабораторной работы

2.1 Установка и первичная настройка MariaDB

На виртуальной машине server открываю терминал и перехожу в режим суперпользователя. Для установки сервера баз данных MariaDB и клиентских утилит выполняю команду:

```
dnf -y install mariadb mariadb-server
```

Пакет mariadb содержит клиентские программы для подключения к серверу MariaDB и работы с базами данных, а пакет mariadb-server содержит серверную часть СУБД. Параметр -y автоматически подтверждает установку пакетов и необходимых зависимостей.

В процессе установки менеджер пакетов dnf устанавливает непосредственно MariaDB, серверные и клиентские компоненты, а также дополнительные зависимости, среди которых mariadb-common, mariadb-client-utils, mariadb-server-utils, mariadb-backup, модули SELinux и библиотеки Perl. Сообщения Transaction test succeeded и Complete! свидетельствуют об успешном завершении установки (см. рис. Рисунок 2.1).

```

Transaction test succeeded.
Running transaction
  Preparing      :                                1/1
  Installing     : mariadb-common-3:10.11.18-1.el10_2.x86_64 1/11
  Installing     : perl-Sys-Hostname-1.25-515.el10_2.x86_64 2/11
  Installing     : mariadb-errmsg-3:10.11.18-1.el10_2.noarch 3/11
  Installing     : perl-DBD-MariaDB-1.23-10.el10.x86_64 4/11
  Running scriptlet: mysql-selinux-1.0.14-1.el10_0.noarch 5/11
  Installing     : mysql-selinux-1.0.14-1.el10_0.noarch 5/11
  Running scriptlet: mysql-selinux-1.0.14-1.el10_0.noarch 5/11
libsemanage.semanage_direct_install_info: Overriding mysql module at lower priority 100 with module at p
riority 200.

  Installing     : mariadb-client-utils-3:10.11.18-1.el10_2.x86_64 6/11
  Installing     : mariadb-backup-3:10.11.18-1.el10_2.x86_64 7/11
  Installing     : mariadb-gssapi-server-3:10.11.18-1.el10_2.x86_64 8/11
  Installing     : mariadb-server-utils-3:10.11.18-1.el10_2.x86_64 9/11
  Running scriptlet: mariadb-server-3:10.11.18-1.el10_2.x86_64 10/11
  Installing     : mariadb-server-3:10.11.18-1.el10_2.x86_64 10/11
  Running scriptlet: mariadb-server-3:10.11.18-1.el10_2.x86_64 10/11
  Installing     : mariadb-3:10.11.18-1.el10_2.x86_64 11/11
  Running scriptlet: mysql-selinux-1.0.14-1.el10_0.noarch 11/11
  Running scriptlet: mariadb-3:10.11.18-1.el10_2.x86_64 11/11

Installed:
mariadb-3:10.11.18-1.el10_2.x86_64      mariadb-backup-3:10.11.18-1.el10_2.x86_64
mariadb-client-utils-3:10.11.18-1.el10_2.x86_64 mariadb-common-3:10.11.18-1.el10_2.noarch
mariadb-errmsg-3:10.11.18-1.el10_2.noarch mariadb-gssapi-server-3:10.11.18-1.el10_2.x86_64
mariadb-server-3:10.11.18-1.el10_2.x86_64 mariadb-server-utils-3:10.11.18-1.el10_2.x86_64
mysql-selinux-1.0.14-1.el10_0.noarch      perl-DBD-MariaDB-1.23-10.el10.x86_64
perl-Sys-Hostname-1.25-515.el10_2.x86_64

Complete!
[root@server.emcikunova.net ~]#

```

Рисунок 2.1: Установка пакетов MariaDB и необходимых зависимостей

После установки просматриваю набор конфигурационных файлов MariaDB в каталоге `/etc/my.cnf.d` и содержимое основного конфигурационного файла `/etc/my.cnf`. Для этого выполняю:

```

ls /etc/my.cnf.d/
cat /etc/my.cnf

```

В каталоге `/etc/my.cnf.d` присутствуют следующие конфигурационные файлы:

- `auth_gssapi.cnf` — параметры модуля аутентификации GSSAPI;
- `client.cnf` — параметры клиентских программ MariaDB;
- `enable_encryption.preset` — набор параметров, относящихся к настройкам шифрования;
- `mariadb-server.cnf` — основные параметры серверной части MariaDB;
- `mysql-clients.cnf` — параметры клиентских программ, совместимых с MySQL;

- `provider_bzip2.cnf`, `provider_lz4.cnf`, `provider_lzo.cnf` и `provider_snappy.cnf` — конфигурационные фрагменты провайдеров соответствующих алгоритмов сжатия;
- `spider.cnf` — параметры механизма хранения Spider.

Основной файл `/etc/my.cnf` имеет следующее содержимое:

```
#
# This group is read both both by the client and the server
# use it for options that affect everything
#
[client-server]

#
# include all files from the config directory
#
!includedir /etc/my.cnf.d
```

Строки, начинающиеся с символа `#`, являются комментариями и не влияют на работу MariaDB. Первые комментарии поясняют, что следующая группа параметров используется как клиентскими программами, так и сервером.

Строка:

```
[client-server]
```

объявляет секцию общих параметров, применяемых одновременно к клиентской и серверной частям MariaDB.

Следующие строки-комментарии указывают, что основная конфигурация дополняется файлами из отдельного каталога.

Директива:

```
!includedir /etc/my.cnf.d
```

предписывает MariaDB загружать дополнительные конфигурационные файлы из каталога `/etc/my.cnf.d`. Благодаря этому настройки можно разделять между несколькими файлами, не изменяя основной `/etc/my.cnf` (см. рис. Рисунок 2.2).

```
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# ls /etc/my.cnf.d/  
auth_gssapi.cnf  enable_encryption.preset  mysql-clients.cnf  provider_lz4.cnf  provider_snappy.cnf  
client.cnf      mariadb-server.cnf          provider_bzip2.cnf  provider_lzo.cnf  spider.cnf  
[root@server.emcikunova.net ~]# cat /etc/my.cnf  
#  
# This group is read both both by the client and the server  
# use it for options that affect everything  
#  
[client-server]  
  
#  
# include all files from the config directory  
#  
!includedir /etc/my.cnf.d  
[root@server.emcikunova.net ~]#
```

Рисунок 2.2: Просмотр конфигурационных файлов MariaDB

После установки запускаю службу MariaDB и настраиваю её автоматический запуск вместе с операционной системой:

```
systemctl start mariadb  
systemctl enable mariadb
```

Команда `systemctl start mariadb` запускает сервер базы данных в текущем сеансе работы системы. Команда `systemctl enable mariadb` добавляет службу в автозагрузку. Сообщение о создании символической ссылки в каталоге `multi-user.target.wants` подтверждает включение автоматического запуска MariaDB.

Для проверки открытого сервером сетевого порта выполняю:

```
ss -tulpen | grep mysql
```

Поскольку имя процесса в установленной версии отображается как `mariabdd`, дополнительно использую:

```
ss -tulpen | grep mariadb
```

В результате видно, что процесс `mariabdd` прослушивает TCP-порт 3306 как для IPv4, так и для IPv6. Адрес `0.0.0.0:3306` означает прослушивание IPv4-интерфейсов, а `:::3306` — IPv6-интерфейсов. Стандартным портом сервера MariaDB/MySQL является TCP-порт 3306 (см. рис. Рисунок 2.3).

```
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# systemctl start mariadb  
[root@server.emcikunova.net ~]# systemctl enable mariadb  
Created symlink '/etc/systemd/system/mysql.service' → '/usr/lib/systemd/system/mariadb.service'.  
Created symlink '/etc/systemd/system/mysqld.service' → '/usr/lib/systemd/system/mariadb.service'.  
Created symlink '/etc/systemd/system/multi-user.target.wants/mariadb.service' → '/usr/lib/systemd/system/mariadb.service'.  
[root@server.emcikunova.net ~]# ss -tulpen | grep mysql  
[root@server.emcikunova.net ~]# ss -tulpen | grep mariadb  
tcp    LISTEN 0      80          0.0.0.0:3306      0.0.0.0:*      users:((("mariabdd",p  
id=16388,fd=17))  
    uid:27 ino:61650 sk:19 cgroup:/system.slice/mariadb.service <->  
tcp    LISTEN 0      80          :::3306          :::*          users:((("mariabdd",p  
id=16388,fd=18))  
    uid:27 ino:61651 sk:25 cgroup:/system.slice/mariadb.service v6only:1 <->  
[root@server.emcikunova.net ~]#
```

Рисунок 2.3: Запуск MariaDB, включение автозапуска и проверка порта 3306

Для выполнения первоначальной настройки безопасности запускаю интерактивный скрипт:

```
mysql_secure_installation
```

В начале работы скрипт запрашивает текущий пароль пользователя `root` MariaDB. Поскольку выполняется первоначальная настройка, продолжаю настройку и устанавливаю пароль администратора базы данных.

На предложение переключить пользователя `root` на аутентификацию через Unix-сокет:

```
Switch to unix_socket authentication [Y/n]
```

выбираю `n`, поскольку для дальнейшего выполнения лабораторной работы требуется вход по паролю с использованием команды `mysql -u root -p`.

На запрос изменения пароля пользователя `root` задаю новый пароль. Сообщение `Password updated successfully!` подтверждает успешную установку па-

роля, а Reloading privilege tables.. Success! — успешное обновление таблиц привилегий (см. рис. Рисунок 2.4).

```
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# mysql_secure_installation  
  
NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB  
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!  
  
In order to log into MariaDB to secure it, we'll need the current  
password for the root user. If you've just installed MariaDB, and  
haven't set the root password yet, you should just press enter here.  
  
Enter current password for root (enter for none):  
OK, successfully used password, moving on...  
  
Setting the root password or using the unix_socket ensures that nobody  
can log into the MariaDB root user without the proper authorisation.  
  
You already have your root account protected, so you can safely answer 'n'.  
  
Switch to unix_socket authentication [Y/n] n  
... skipping.  
  
You already have your root account protected, so you can safely answer 'n'.  
  
Change the root password? [Y/n]  
New password:  
Re-enter new password:  
Password updated successfully!  
Reloading privilege tables..  
... Success!
```

Рисунок 2.4: Установка пароля администратора MariaDB

Далее скрипт предлагает удалить анонимных пользователей. Подтверждаю операцию, чтобы исключить возможность подключения к MariaDB без заранее созданной учётной записи.

На запрос:

```
Disallow root login remotely? [Y/n]
```

также отвечаю утвердительно. В результате пользователь root MariaDB может использоваться для администрирования локально, а удалённый вход с этой учётной записью запрещается.

Также подтверждаю удаление тестовой базы данных test и прав доступа к ней. В завершение выполняется повторная загрузка таблиц привилегий, благодаря чему внесённые изменения начинают действовать сразу. Все операции завершаются сообщениями Success! (см. рис. Рисунок 2.5).

```

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n]
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n]
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n]
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n]
... Success!

Cleaning up...

```

Рисунок 2.5: Завершение настройки безопасности MariaDB

После настройки безопасности вхожу в интерактивную оболочку MariaDB с правами администратора:

```
mysql -u root -p
```

Параметр `-u root` указывает имя пользователя базы данных, а `-p` сообщает клиенту о необходимости запросить пароль.

Для просмотра справки и списка команд интерактивного клиента выполняю:

```
\h
```

В результате MariaDB выводит перечень команд клиента. В частности, `help` используется для вызова справки, `status` — для получения информации о текущем соединении и сервере, `use` — для выбора базы данных, `source` — для выполнения SQL-команд из файла, а `quit` и `exit` — для завершения работы с интерактивной оболочкой (см. рис. Рисунок 2.6).

```
http://mariadb.org

List of all client commands:
Note that all text commands must be first on line and end with ';'
?      (\?) Synonym for 'help'.
charset (\C) Switch to another charset. Might be needed for processing binlog with multi-byte charsets
.
clear  (\c) Clear the current input statement.
connect (\r) Reconnect to the server. Optional arguments are db and host.
delimiter (\d) Set statement delimiter.
edit    (\e) Edit command with $EDITOR.
ego     (\G) Send command to MariaDB server, display result vertically.
exit    (\q) Exit mysql. Same as quit.
go      (\g) Send command to MariaDB server.
help    (\h) Display this help.
nopager (\n) Disable pager, print to stdout.
notee   (\t) Don't write into outfile.
nowarning (\W) Don't show warnings after every statement.
pager   (\P) Set PAGER [to_pager]. Print the query results via PAGER.
print   (\p) Print current command.
prompt  (\R) Change your mysql prompt.
quit    (\q) Quit mysql.
rehash  (\#) Rebuild completion hash.
sandbox (\-) Disallow commands that access the file system (except \P without an argument and \e).
source  (\.) Execute an SQL script file. Takes a file name as an argument.
status  (\s) Get status information from the server.
system  (\!) Execute a system shell command.
tee      (\T) Set outfile [to_outfile]. Append everything into given outfile.
use      (\u) Use another database. Takes database name as argument.
warnings (\W) Show warnings after every statement.

For server side help, type 'help contents'

MariaDB [(none)]> |
```

Рисунок 2.6: Просмотр справки интерактивной оболочки MariaDB

Для просмотра имеющихся баз данных выполняю SQL-запрос:

```
SHOW DATABASES;
```

После первоначальной настройки в системе присутствуют четыре системные базы данных:

- `information_schema` — виртуальная база с метаданными о базах данных, таблицах, столбцах, правах доступа и других объектах СУБД;
- `mysql` — системная база MariaDB, содержащая сведения о пользователях, привилегиях и внутренних настройках сервера;
- `performance_schema` — база, предоставляющая сведения о производительности и внутренних событиях сервера;
- `sys` — набор представлений и вспомогательных объектов, упрощающих анализ диагностических данных MariaDB.

Тестовая база `test` в списке отсутствует, поскольку она была удалена при выполнении `mysql_secure_installation`.

Для выхода из интерактивной оболочки использую:

```
exit;
```

После этого управление возвращается в командную оболочку операционной системы (см. рис. Рисунок 2.7).

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.000 sec)

MariaDB [(none)]> exit;
Bye
[root@server.emcikunova.net ~]#
```

Рисунок 2.7: Просмотр системных баз данных MariaDB

2.2 Конфигурация кодировки символов UTF-8

Перед изменением кодировки повторно вхожу в MariaDB с правами администратора:

```
mysql -u root -p
```

Для просмотра состояния текущего соединения выполняю:

```
status
```

До изменения конфигурации команда выводит информацию о клиенте и сервере MariaDB (см. рис. Рисунок 2.8).

Первая строка:

```
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using EditLine wrapper
```

показывает версию клиентской программы — 15.1, версию MariaDB — 10.11.18, архитектуру x86_64 и использование библиотеки EditLine для обработки ввода.

Строка:

```
Connection id: 12
```

содержит идентификатор текущего соединения с сервером.

Поле:

```
Current database:
```

пустое, поскольку после подключения конкретная пользовательская база данных ещё не выбрана.

Строка:

```
Current user: root@localhost
```

означает, что соединение выполнено от имени пользователя root локально.

Строка:

```
SSL: Not in use
```

показывает, что для данного локального соединения шифрование SSL/TLS не используется.

Строка:

```
Current pager: stdout
```

означает, что результаты запросов выводятся непосредственно на стандартный вывод терминала.

Строка:

```
Using outfile: ''
```

показывает, что вывод результатов в отдельный файл не настроен.

Строка:

```
Using delimiter: ;
```

указывает, что символ ; является стандартным разделителем, завершающим SQL-запросы.

Строка:

```
Server: MariaDB
```

определяет используемый сервер баз данных.

Строка:

```
Server version: 10.11.18-MariaDB MariaDB Server
```

показывает установленную версию серверной части MariaDB.

Строка:

```
Protocol version: 10
```

содержит версию сетевого протокола MySQL/MariaDB.

Строка:

```
Connection: Localhost via UNIX socket
```

показывает, что локальное соединение с сервером выполняется через UNIX-сокет, а не через TCP.

До изменения конфигурации отображаются следующие кодировки:

```
Server charsetset: latin1
Db      charsetset: latin1
Client charsetset: utf8mb3
Conn.  charsetset: utf8mb3
```

Следовательно, кодировкой сервера и баз данных по умолчанию является `latin1`, в то время как клиент и текущее соединение используют `utf8mb3`.

Строка:

```
UNIX socket: /var/lib/mysql/mysql.sock
```

содержит путь к UNIX-сокету MariaDB.

Поле `Uptime` показывает время, прошедшее с момента запуска сервера. В нижней строке также выводится служебная статистика: количество активных потоков, обработанных запросов, медленных запросов, открытий таблиц, открытых таблиц и среднее число запросов в секунду.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> status
-----
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using EditLine wrapper

Connection id:          12
Current database:
Current user:           root@localhost
SSL:                   Not in use
Current pager:          stdout
Using outfile:          ''
Using delimiter:        ;
Server:                 MariaDB
Server version:         10.11.18-MariaDB MariaDB Server
Protocol version:       10
Connection:             Localhost via UNIX socket
Server charsetset:      latin1
Db      charsetset:     latin1
Client charsetset:      utf8mb3
Conn.  charsetset:      utf8mb3
UNIX socket:            /var/lib/mysql/mysql.sock
Uptime:                 5 min 43 sec

Threads: 1  Questions: 23  Slow queries: 0  Opens: 20  Open tables: 13  Queries per second avg: 0.067
-----
MariaDB [(none)]>
```

Рисунок 2.8: Статус MariaDB до изменения кодировки

Для изменения кодировки по умолчанию перехожу в каталог `/etc/my.cnf.d` и создаю отдельный конфигурационный файл `utf8.cnf`:

```
cd /etc/my.cnf.d  
touch utf8.cnf
```

В файл записываю:

```
[client]  
default-character-set = utf8  
  
[mysqld]  
character-set-server = utf8
```

Секция [client] содержит параметры клиентских программ. Директива:

```
default-character-set = utf8
```

устанавливает UTF-8 в качестве кодировки по умолчанию для клиентов MariaDB.

Секция [mysqld] относится непосредственно к серверному процессу MariaDB. Директива:

```
character-set-server = utf8
```

задаёт UTF-8 в качестве кодировки сервера по умолчанию. Содержимое созданного конфигурационного файла показано на рис. Рисунок 2.9.

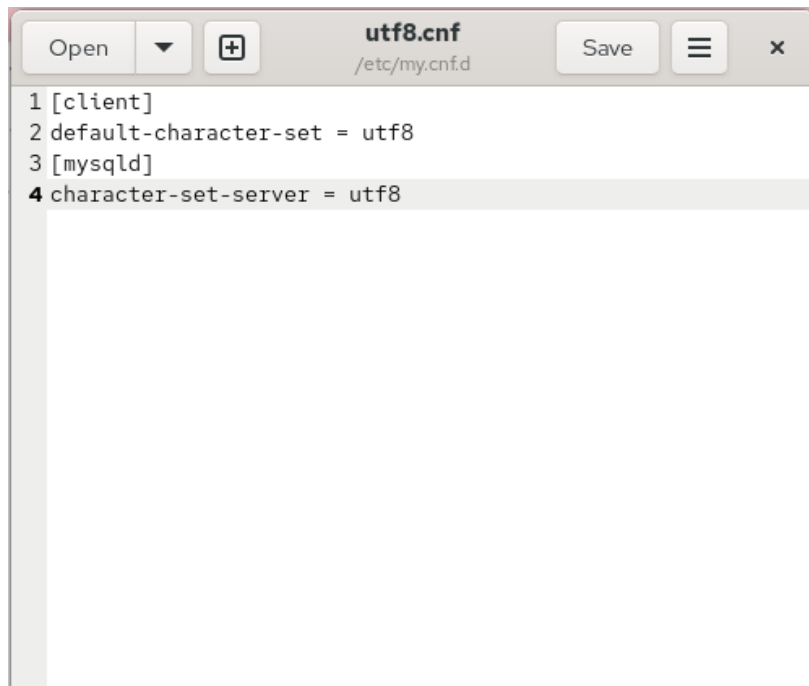


Рисунок 2.9: Содержимое конфигурационного файла utf8.cnf

Для применения новых параметров перезапускаю MariaDB:

```
systemctl restart mariadb
```

После перезапуска снова подключаюсь к серверу:

```
mysql -u root -p
```

и выполняю:

```
status
```

После внесения изменений значения кодировок имеют следующий вид:

```
Server characterset: utf8mb3
Db      characterset: utf8mb3
Client characterset: utf8mb3
Conn.   characterset: utf8mb3
```

В MariaDB 10.11 имя `utf8` соответствует трёхбайтовой реализации UTF-8 и в статусе отображается под названием `utf8mb3`. До изменения параметров сервер и база данных использовали `latin1`, а после перезапуска все четыре параметра — сервер, база данных, клиент и соединение — используют `utf8mb3`. Это подтверждает успешное применение созданного файла `utf8.cnf` (см. рис. Рисунок 2.10).

Остальные поля `status`, например идентификатор соединения, время работы сервера и количество выполненных запросов, изменились вследствие перезапуска MariaDB и создания нового соединения и не относятся непосредственно к изменению кодировки.

```
MariaDB [(none)]> status
-----
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using Editline wrapper

Connection id:          3
Current database:
Current user:           root@localhost
SSL:                    Not in use
Current pager:          stdout
Using outfile:          ''
Using delimiter:        ;
Server:                 MariaDB
Server version:         10.11.18-MariaDB MariaDB Server
Protocol version:       10
Connection:             Localhost via UNIX socket
Server characterset:     utf8mb3
Db characterset:         utf8mb3
Client characterset:     utf8mb3
Conn. characterset:      utf8mb3
UNIX socket:            /var/lib/mysql/mysql.sock
Uptime:                 22 sec

Threads: 1  Questions: 4  Slow queries: 0  Opens: 17  Open tables: 10  Queries per second avg: 0.181
-----
MariaDB [(none)]> █
```

Рисунок 2.10: Статус MariaDB после настройки кодировки UTF-8

2.3 Создание базы данных `addressbook`

Подключившись к MariaDB от имени администратора, создаю базу данных `addressbook`:

```
CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
```

Параметр `CHARACTER SET utf8` задаёт UTF-8 в качестве набора символов базы

данных, а `COLLATE utf8_general_ci` устанавливает правила сравнения и сортировки строк. Суффикс `ci` означает `case insensitive`, то есть сравнение символов без учёта регистра.

После успешного создания выбираю базу данных для дальнейшей работы:

```
USE addressbook;
```

Проверяю список таблиц:

```
SHOW TABLES;
```

На этом этапе MariaDB возвращает:

```
Empty set
```

поскольку база только создана и ещё не содержит таблиц.

Создаю таблицу `city`:

```
CREATE TABLE city(  
    name VARCHAR(40),  
    city VARCHAR(40)  
);
```

Таблица состоит из двух текстовых полей. Поле `name` предназначено для фамилии сотрудника, а `city` — для города, в котором он работает. Для обоих полей используется тип `VARCHAR(40)`, позволяющий хранить строки переменной длины до 40 символов.

Затем добавляю сведения о трёх сотрудниках:

```
INSERT INTO city(name,city) VALUES ('Иванов','Москва');  
INSERT INTO city(name,city) VALUES ('Петров','Сочи');  
INSERT INTO city(name,city) VALUES ('Сидоров','Дубна');
```

Каждое сообщение:

```
Query OK, 1 row affected
```

означает, что соответствующая запись успешно добавлена в таблицу.

Для проверки содержимого выполняю:

```
SELECT * FROM city;
```

Результат запроса содержит три строки:

Иванов	Москва
Петров	Сочи
Сидоров	Дубна

Символ `*` в запросе `SELECT` означает выбор всех столбцов таблицы. Корректное отображение кириллических фамилий и названий городов также подтверждает работу настроенной кодировки UTF-8 (см. рис. Рисунок 2.11).

```
mariaDB [(none)]>
MariaDB [(none)]> CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE addressbook;
Database changed
MariaDB [addressbook]> SHOW TABLES;
Empty set (0.000 sec)

MariaDB [addressbook]> CREATE TABLE city(name VARCHAR(40), city VARCHAR(40));
Query OK, 0 rows affected (0.013 sec)

MariaDB [addressbook]> INSERT INTO city(name,city) VALUES ('Иванов','Москва');
Query OK, 1 row affected (0.001 sec)

MariaDB [addressbook]> INSERT INTO city(name,city) VALUES ('Петров','Сочи');
Query OK, 1 row affected (0.001 sec)

MariaDB [addressbook]> INSERT INTO city(name,city) VALUES ('Сидоров','Дубна');
Query OK, 1 row affected (0.002 sec)

MariaDB [addressbook]> SELECT * FROM city;
+-----+-----+
| name      | city      |
+-----+-----+
| Иванов    | Москва    |
| Петров    | Сочи      |
| Сидоров   | Дубна     |
+-----+-----+
3 rows in set (0.000 sec)

MariaDB [addressbook]>
```

Рисунок 2.11: Создание базы addressbook, таблицы city и добавление данных

Для работы с базой данных создаю отдельного пользователя с логином emcikunova:

```
CREATE USER emcikunova@'%' IDENTIFIED BY '123456';
```

Имя пользователя состоит из двух частей. emcikunova является именем учётной записи MariaDB, а символ % в части хоста означает возможность сопоставления подключения с различных адресов. Фактическая возможность удалённого подключения дополнительно зависит от сетевой конфигурации сервера и правил межсетевого экрана.

Далее предоставляю созданному пользователю права на просмотр, добавление, изменение и удаление записей базы addressbook:

```
GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO emcikunova@'%';
```

Здесь:

- SELECT предоставляет право чтения данных;
- INSERT — добавления новых записей;
- UPDATE — изменения существующих записей;
- DELETE — удаления записей;
- addressbook.* означает применение перечисленных прав ко всем таблицам базы addressbook.

Для применения изменений в таблицах привилегий выполняю:

```
FLUSH PRIVILEGES;
```

После этого просматриваю структуру таблицы:

```
DESCRIBE city;
```

Результат показывает наличие двух полей:

```
name    varchar(40)
city    varchar(40)
```

Для обоих полей значение Null равно YES, следовательно, поля допускают значение NULL. Значение Default равно NULL, первичные или иные ключи для таблицы не определены, а дополнительные атрибуты отсутствуют.

Завершаю работу с интерактивной оболочкой:

```
quit;
```

(см. рис. Рисунок 2.12).

```
MariaDB [addressbook]>
MariaDB [addressbook]> CREATE USER emcikunova@%' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.001 sec)

MariaDB [addressbook]> GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO emcikunova@%';
Query OK, 0 rows affected (0.002 sec)

MariaDB [addressbook]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [addressbook]> DESCRIBE city;
+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| name  | varchar(40) | YES  |     | NULL    |       |
| city  | varchar(40) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)

MariaDB [addressbook]> quit;
Bye
```

Рисунок 2.12: Создание пользователя emcikunova и назначение прав на addressbook

После выхода из MariaDB проверяю список баз данных с помощью отдельной клиентской утилиты:

```
mysqlshow -u root -p
```

В списке теперь присутствуют:

```
addressbook
information_schema
mysql
performance_schema
sys
```

Наличие addressbook подтверждает успешное создание пользовательской базы.

Для просмотра таблиц базы addressbook от имени администратора выполняю:

```
mysqlshow -u root -p addressbook
```

Утилита выводит таблицу city.

Дополнительно проверяю доступ от имени ранее созданного пользователя:

```
mysqlshow -u emcikunova -p addressbook
```

После ввода пароля пользователь emcikunova также получает список таблиц базы addressbook, содержащий city. Это подтверждает корректную работу созданной учётной записи и назначенных ей прав доступа (см. рис. Рисунок 2.13).

```

[root@server.emcikunova.net my.cnf.d]# mysqlshow -u root -p
Enter password:
+-----+
| Databases |
+-----+
| addressbook |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
[root@server.emcikunova.net my.cnf.d]#
[root@server.emcikunova.net my.cnf.d]# mysqlshow -u root -p addressbook
Enter password:
Database: addressbook
+-----+
| Tables |
+-----+
| city |
+-----+
[root@server.emcikunova.net my.cnf.d]# mysqlshow -u emcikunova -p addressbook
Enter password:
Database: addressbook
+-----+
| Tables |
+-----+
| city |
+-----+
[root@server.emcikunova.net my.cnf.d]#

```

Рисунок 2.13: Проверка базы addressbook и таблицы city с помощью mysqlshow

2.4 Создание и восстановление резервных копий базы данных

Для хранения резервных копий создаю каталог /var/backup:

```
mkdir -p /var/backup
```

Параметр -p создаёт каталог при его отсутствии и не выдаёт ошибку, если он уже существует.

Несжатую резервную копию базы addressbook создаю командой:

```
mysqldump -u root -p addressbook > /var/backup/addressbook.sql
```

Утилита mysqldump формирует SQL-представление структуры и содержимого базы. Параметр -u root задаёт пользователя, -p включает запрос пароля, а addressbook определяет базу, содержимое которой необходимо

сохранить. Оператор `>` перенаправляет сформированный SQL-дамп в файл `/var/backup/addressbook.sql`.

Для экономии дискового пространства создаю сжатую резервную копию:

```
mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz
```

В данном случае стандартный вывод `mysqldump` передаётся через конвейер `|` программе `gzip`, после чего сжатые данные записываются в файл `addressbook.sql.gz`.

Также создаю резервную копию, имя которой содержит дату и время:

```
mysqldump -u root -p addressbook | gzip > $(date +%Y%m%d.%H%M%S).sql.gz
```

Команда `date` формирует имя файла, в котором:

- `%Y` — четырёхзначный год;
- `%m` — месяц;
- `%d` — день;
- `%H` — часы;
- `%M` — минуты;
- `%S` — секунды.

В результате каждая резервная копия может получать уникальное имя, отражающее момент её создания.

Для проверки возможности восстановления сначала восстанавливаю базу из обычного SQL-файла:

```
mysql -u root -p addressbook < /var/backup/addressbook.sql
```

Оператор `<` передаёт содержимое файла `addressbook.sql` на стандартный ввод клиента `mysql`, который выполняет содержащиеся в дампе SQL-команды в базе `addressbook`.

Затем проверяю восстановление из сжатого файла:

```
zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook
```

Команда `zcat` распаковывает содержимое архива в стандартный вывод без создания промежуточного несжатого файла, после чего SQL-команды передаются через конвейер клиенту `mysql`.

Возврат приглашения командной строки без сообщений об ошибках после обеих операций свидетельствует об успешной обработке файлов резервных копий (см. рис. Рисунок 2.14).

```
[root@server.emcikunova.net my.cnf.d]#  
[root@server.emcikunova.net my.cnf.d]# mkdir -p /var/backup  
[root@server.emcikunova.net my.cnf.d]# mysqldump -u root -p addressbook > /var/backup/addressbook.sql  
Enter password:  
[root@server.emcikunova.net my.cnf.d]# mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz  
Enter password:  
[root@server.emcikunova.net my.cnf.d]# mysqldump -u root -p addressbook | gzip > $(date +%Y%m%d.%H%M%S).sql.gz  
Enter password:  
[root@server.emcikunova.net my.cnf.d]#  
[root@server.emcikunova.net my.cnf.d]# mysql -u root -p addressbook < /var/backup/addressbook.sql  
Enter password:  
[root@server.emcikunova.net my.cnf.d]# zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook  
Enter password:  
[root@server.emcikunova.net my.cnf.d]#
```

Рисунок 2.14: Создание и восстановление резервных копий базы `addressbook`

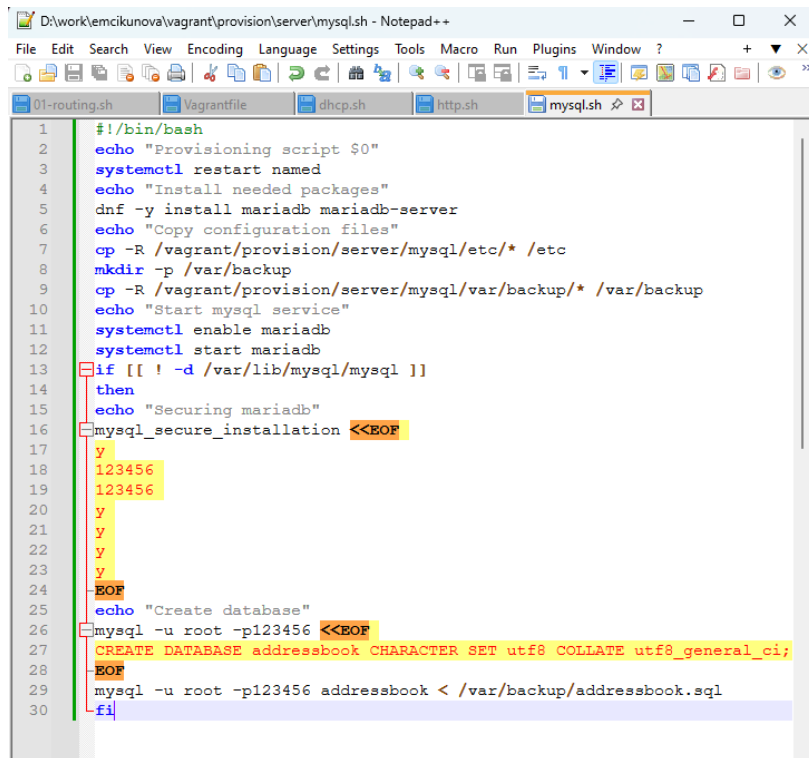
2.5 Автоматизация установки и настройки MariaDB средствами Vagrant

Для фиксации выполненных действий в конфигурации внутреннего окружения виртуальной машины перехожу в каталог серверного `provision`:

```
[root@server.emcikunova.net my.cnf.d]#  
[root@server.emcikunova.net my.cnf.d]# cd /vagrant/provision/server/  
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/mysql/etc/my.cnf.d  
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/mysql/var/backup  
[root@server.emcikunova.net server]# cp -R /etc/my.cnf.d/utf8.cnf /vagrant/provision/server/mysql/etc/my.cnf.d/  
[root@server.emcikunova.net server]# cp -R /var/backup/* /vagrant/provision/server/mysql/var/backup/  
[root@server.emcikunova.net server]# cd /vagrant/provision/server/  
[root@server.emcikunova.net server]# touch mysql.sh  
[root@server.emcikunova.net server]# chmod +x mysql.sh  
[root@server.emcikunova.net server]#
```

Рисунок 2.15: Подготовка файлов для автоматической настройки MariaDB

Содержимое созданного файла `mysql.sh` показано на рис. Рисунок 2.16.



```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  systemctl restart named
4  echo "Install needed packages"
5  dnf -y install mariadb mariadb-server
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/mysql/etc/* /etc
8  mkdir -p /var/backup
9  cp -R /vagrant/provision/server/mysql/var/backup/* /var/backup
10 echo "Start mysql service"
11 systemctl enable mariadb
12 systemctl start mariadb
13 if [[ ! -d /var/lib/mysql/mysql ]]
14 then
15     echo "Securing mariadb"
16     mysql_secure_installation <<EOF
17     y
18     123456
19     123456
20     y
21     y
22     y
23     y
24     EOF
25     echo "Create database"
26     mysql -u root -p123456 <<EOF
27     CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
28     EOF
29     mysql -u root -p123456 addressbook < /var/backup/addressbook.sql
30 fi
```

Рисунок 2.16: Скрипт автоматической установки и настройки MariaDB

3 Вывод

В ходе работы установлена и настроена MariaDB. Выполнена настройка безопасности, установлена кодировка UTF-8, создана база данных addressbook с таблицей city и отдельным пользователем.

Также созданы резервные копии базы данных и выполнено их восстановление. Настройка MariaDB автоматизирована с помощью скрипта `mysql.sh`, который подключён в `Vagrantfile`.

4 Контрольные вопросы

1. Какая команда отвечает за настройки безопасности в MariaDB?

Для настройки безопасности используется команда:

```
mysql_secure_installation
```

Она позволяет установить пароль root, удалить анонимных пользователей и тестовую базу, а также запретить удалённый вход для root.

2. Как настроить MariaDB для доступа через сеть?

Необходимо настроить прослушивание сетевого интерфейса, например:

```
[mysqld]
```

```
bind-address=0.0.0.0
```

Также нужно создать пользователя для удалённого подключения и выдать ему права:

```
CREATE USER user@'%' IDENTIFIED BY 'password';
```

```
GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO user@'%';
```

После изменения конфигурации MariaDB перезапускается:

```
systemctl restart mariadb
```

3. Какая команда позволяет получить обзор доступных баз данных после входа в среду оболочки MariaDB?

Используется команда:

```
SHOW DATABASES;
```

Она выводит список доступных баз данных.

4. Какая команда позволяет узнать, какие таблицы доступны в базе данных?

После выбора базы командой:

```
USE addressbook;
```

список таблиц выводится командой:

```
SHOW TABLES;
```

5. Какая команда позволяет узнать, какие поля доступны в таблице?

Для просмотра структуры таблицы используется:

```
DESCRIBE city;
```

Команда показывает имена полей, их типы и свойства.

6. Какая команда позволяет узнать, какие записи доступны в таблице?

Для просмотра всех записей используется:

```
SELECT * FROM city;
```

7. Как удалить запись из таблицы?

Для удаления записи применяется команда DELETE, например:

```
DELETE FROM city WHERE name='Петров';
```

Условие WHERE указывает, какую запись необходимо удалить.

8. Где расположены файлы конфигурации MariaDB? Что можно настроить с их помощью?

Основной файл конфигурации находится по пути:

```
/etc/my.cnf
```

Дополнительные файлы находятся в:

```
/etc/my.cnf.d/
```

В них можно настроить кодировку, сетевые параметры, порт, каталог данных и другие параметры MariaDB.

9. Где располагаются файлы с базами данных MariaDB?

По умолчанию файлы MariaDB хранятся в каталоге:

```
/var/lib/mysql/
```

10. Как сделать резервную копию базы данных и затем её восстановить?

Резервная копия создаётся командой:

```
mysqldump -u root -p addressbook > /var/backup/addressbook.sql
```

Восстановление выполняется командой:

```
mysql -u root -p addressbook < /var/backup/addressbook.sql
```

Для сжатой копии можно использовать:

```
mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz
```

и восстановить её командой:

```
zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook
```